

THINKBOARD

A Complete A to Z Guide of the MERN Stack Application

Prepared for Project Demonstration and Review

Thinkboard: A Complete A to Z Guide of the MERN Stack Application

Welcome to the official **Thinkboard** development guide. This document is a comprehensive, step-by-step manual designed to explain every component, installation step, and file of the Thinkboard application. Whether you are explaining this project to an instructor or deploying it to production, this guide covers everything you need to know.

Table of Contents

- 1. **System Concept & Architecture**
- 2. **System Requirements & Installation**
- 3. **MongoDB Atlas (Database) Setup**
- 4. **Backend Deep-Dive (`/backend`)**
 - Dependencies and Configuration
 - Database Connection Setup
 - Mongoose Data Model
 - Controller Logic (CRUD)
 - Routing
 - Server Configurations & Security
- 5. **Frontend Deep-Dive (`/frontend`)**
 - Styling System (Tailwind & DaisyUI)
 - Main App Shell & Routing
 - Navbar Component
 - Homepage Listing
 - Creating Notes
 - Editing & Details View
- 6. **API Communication & CORS**
- 7. **Running the Project Locally**
- 8. **Deployment to Production (Render)**

1. System Concept & Architecture

Thinkboard is a full-stack **MERN** application. MERN stands for:

- **M**ongoDB: A document-based NoSQL database used to store notes.
- **E**xpress: A web application framework for Node.js used to build backend APIs.
- **R**each: A frontend library used to build the interactive User Interface.
- **N**ode.js: A JavaScript runtime environment used to execute the backend server.

System Architecture Diagram



3. MongoDB Atlas (Database) Setup

Since Thinkboard saves notes in a database, we use **MongoDB Atlas**, a cloud-hosted database. Here is how to set it up:

1. **Sign Up**: Go to [mongodb.com/atlas](https://www.mongodb.com/cloud/atlas/register) and create a free account.
2. **Create Cluster**: Select the **M0 Free** shared cluster option. Choose a cloud provider (like AWS) and a region near you.
3. **Database User Security**:
 - Create a database user.
 - Set a **Username** (e.g., `thinkboard_user`) and a secure **Password**.
 - Write these credentials down; they are needed for your `.env` connection string.*
4. **Network Access**:
 - Go to "Network Access" in the sidebar.
 - Click **Add IP Address** and select **Allow Access From Anywhere** (`0.0.0.0/0`). This is necessary because server deployment hosts like Render change their IP addresses dynamically.
5. **Get Connection String**:
 - Go to the "Database" section.
 - Click **Connect** on your cluster.
 - Select **Drivers** (Node.js).
 - Copy the connection string. It looks like:
`mongodb+srv://<username>:<password>@cluster0.xxxx.mongodb.net/?retryWrites=true&w=majority`
 - Replace `<username>` and `<password>` with your database user credentials.

4. Backend Deep-Dive (`/backend`)

The backend is responsible for database connection, running the API server, validating data, and serving static files in production.

A. Backend Dependencies (`package.json`)

The `package.json` file defines all libraries used on the server.

```
{
  "name": "backend",
  "version": "1.0.0",
  "type": "module",
  "main": "src/server.js",
  "scripts": {
    "dev": "nodemon src/server.js",
    "start": "node src/server.js"
  },
  "dependencies": {
    "cors": "^2.8.6",
    "dotenv": "^17.4.2",
    "express": "^5.2.1",
    "express-rate-limit": "^8.5.2",
    "mongoose": "^9.4.1"
  },
  "devDependencies": {
```

```

    "nodemon": "^3.1.14"
  }
}

```

- `type: "module"`: Enables modern ES Module imports (`import` / `export`) instead of older CommonJS (`require`).
- `express`: The core framework for API routing and request handling.
- `mongoose`: Object Data Modeling (ODM) library for MongoDB.
- `cors`: Enables Cross-Origin Resource Sharing so the React frontend can talk to the server.
- `dotenv`: Loads configuration environment variables from a `.env` file.
- `express-rate-limit`: Security middleware that limits the number of requests per IP to prevent spam or DDoS attacks.
- `nodemon`: Developer utility that restarts the server automatically whenever you modify backend files.

B. Database Connection Setup (`src/config/db.js`)

This module configures Mongoose to connect to our MongoDB instance.

```

import mongoose from "mongoose";

// Async function to connect to MongoDB cloud database
export const connectDB = async () => {
  try {
    await mongoose.connect(process.env.MONGO_URI);
    console.log("Connected to MongoDB successfully");
  } catch (error) {
    console.error("Error connecting to MongoDB:", error);
    process.exit(1); // Exit process with failure code
  }
};

```

- `mongoose.connect()`: Connects to the database using the connection string stored in environment variables.
- `process.exit(1)`: Shuts down the backend immediately if the database is unreachable, preventing the app from running in an unstable state.

C. Mongoose Data Model (`src/models/Note.js`)

This file defines the schema structure of a "Note" document inside MongoDB.

```

import mongoose from 'mongoose';

// Definition of the note schema
const noteSchema = new mongoose.Schema({
  title: {
    type: String,
    required: true // Title is mandatory
  },
  content: {
    type: String,
    required: true // Content is mandatory
  }
}, {
  timestamps: true // Automatically adds 'createdAt' and 'updatedAt' fields
});

const Note = mongoose.model('Note', noteSchema);
export default Note;

```

- `timestamps: true`: Automatically logs when a note is created (`createdAt`) and last updated (`updatedAt`). Mongoose handles this on database operations.

D. CRUD Controller Logic (`src/controllers/notesController.js`)

Contains the application logic for handling notes endpoints.

```
import Note from "../models/Note.js";

// 1. Fetch all notes from database (sorted by newest first)
export async function getAllNotes(req, res) {
  try {
    const notes = await Note.find().sort({ createdAt: -1 });
    res.status(200).json(notes);
  } catch (error) {
    console.error("Error in getAllNotes controller:", error);
    res.status(500).json({ message: "Internal server error" });
  }
}

// 2. Fetch a single note by database ID
export async function getNoteById(req, res) {
  try {
    const note = await Note.findById(req.params.id);
    if (!note) {
      return res.status(404).json({ message: "Note not found" });
    }
    res.status(200).json(note);
  } catch (error) {
    console.error("Error in getNoteById controller:", error);
    res.status(500).json({ message: "Internal server error" });
  }
}

// 3. Create a new note
export async function createNote(req, res) {
  try {
    const { title, content } = req.body;
    // Basic validation
    if (!title || !content) {
      return res.status(400).json({ message: "Title and content are required" });
    }
    const note = new Note({ title, content });
    const savedNote = await note.save();
    res.status(201).json(savedNote);
  } catch (error) {
    console.error("Error in createNote controller:", error);
    res.status(500).json({ message: "Internal server error" });
  }
}

// 4. Update an existing note
export async function updateNote(req, res) {
  try {
    const { title, content } = req.body;
    const note = await Note.findById(req.params.id);
    if (!note) {
      return res.status(404).json({ message: "Note not found" });
    }
    // Update fields if provided
    if (title !== undefined) note.title = title;
    if (content !== undefined) note.content = content;

    const updatedNote = await note.save();
    res.status(200).json(updatedNote);
  }
}
```

```

    } catch (error) {
      console.error("Error in updateNote controller:", error);
      res.status(500).json({ message: "Internal server error" });
    }
  }

// 5. Delete a note
export async function deleteNote(req, res) {
  try {
    const note = await Note.findByIdAndDelete(req.params.id);
    if (!note) {
      return res.status(404).json({ message: "Note not found" });
    }
    res.status(200).json({ message: "Note deleted successfully" });
  } catch (error) {
    console.error("Error in deleteNote controller:", error);
    res.status(500).json({ message: "Internal server error" });
  }
}

```

E. Backend Routing (`src/routes/notesRoutes.js`)

Maps HTTP methods and URLs to controller functions.

```

import express from 'express';
import { getAllNotes, getNoteById, createNote, updateNote, deleteNote } from '../controllers/notesController.js';

const router = express.Router();

// Route mappings
router.get("/", getAllNotes);           // GET /api/notes
router.get("/:id", getNoteById);        // GET /api/notes/:id
router.post("/", createNote);           // POST /api/notes
router.put("/:id", updateNote);         // PUT /api/notes/:id
router.delete("/:id", deleteNote);      // DELETE /api/notes/:id

export default router;

```

F. Server Entry Point (`src/server.js`)

Assembles and configures the Express application, applies middlewares, connects database, and runs the server listener.

```

import dns from 'dns';
// Force IPv4 routing order to avoid local network resolution issues
dns.setDefaultResultOrder('ipv4first');
dns.setServers(['8.8.8.8', '1.1.1.1']);

import dotenv from 'dotenv';
dotenv.config(); // Loads env variables from .env file

import express from 'express';
import path from 'path';
import cors from 'cors';
import notesRoutes from './routes/notesRoutes.js';
import { connectDB } from './config/db.js';
import { rateLimit } from 'express-rate-limit';

const app = express();
const PORT = process.env.PORT || 3000;

```

```

const __dirname = path.resolve();

// Middlewares
app.use(cors()); // Permits cross-origin resource queries
app.use(express.json()); // Parses JSON body payloads in requests

// Rate Limiter: Stops clients from flooding API endpoints with requests
const limiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutes window
  limit: 100, // Maximum 100 requests per IP address
  standardHeaders: 'draft-7',
  legacyHeaders: false,
  message: { message: "Too many requests from this IP, please try again after 15 minutes." }
});
app.use("/api", limiter);

// Mount Routing
app.use("/api/notes", notesRoutes);

// Root informational endpoint
app.get('/', (req, res) => {
  res.send('Thinkboard API running. Use /api/notes for the notes endpoints.');
```

```

});

// Production Configuration: Serve Built React Assets
if (process.env.NODE_ENV === "production") {
  // Set build folder as static asset host
  app.use(express.static(path.join(__dirname, "../frontend/dist")));

  // Client Routing Fallback: Redirect all unrecognized routes to index.html
  app.use((req, res) => {
    res.sendFile(path.join(__dirname, "../frontend/dist/index.html"));
  });
}

// Start Server Sequence
const start = async () => {
  try {
    await connectDB(); // Ensure DB is connected before starting listener
    app.listen(PORT, () => {
      console.log(`Server is running on port: ${PORT}`);
    });
  } catch (error) {
    console.error('Server startup failed:', error);
    process.exit(1);
  }
};

start();

---
```

5. Frontend Deep-Dive (`/frontend`)

The frontend is built with React 19 and Vite. Styling is handled with Tailwind CSS and DaisyUI using a styled Halloween dark theme.

A. Main App Shell & Routing (`src/App.jsx`)

Sets up routes using React Router and defines the Halloween global UI theme.

```

import { Route, Routes } from 'react-router-dom';
import { Toaster } from 'react-hot-toast';

import Homepage from '../pages/Homepage';
```



```

import CreatePage from './pages/createpage';
import NoteDetailsPage from './pages/NoteDetailsPage';

const App = () => {
  return (
    // 'data-theme="halloween"' applies the DaisyUI Halloween theme color palette
    <div data-theme="halloween" className="min-h-screen bg-base-100 text-base-content">
      <Routes>
        <Route path="/" element={<Homepage />} />
        <Route path="/create" element={<CreatePage />} />
        <Route path="/notes/:id" element={<NoteDetailsPage />} />
      </Routes>
      {/* Toast notifications pop-up container */}
      <Toaster position="top-right" />
    </div>
  );
};

export default App;

```

B. Navbar Component (`src/components/Navbar.jsx`)

The header layout displayed across pages, providing branding and a shortcut to create notes.

```

import { Link } from "react-router-dom";
import { PlusIcon } from "lucide-react";

const Navbar = () => {
  return (
    <header className="bg-base-300 border-b border-base-content/10">
      <div className="mx-auto max-w-6xl p-4">
        <div className="flex items-center justify-between">
          <h1 className="text-3xl font-bold text-primary font-mono tracking-tight">
            Thinkboard
          </h1>
          <div className="flex items-center gap-4">
            <Link to="/create" className="btn btn-primary">
              <PlusIcon className="size-5" />
              <span>New Note</span>
            </Link>
          </div>
        </div>
      </div>
    </header>
  );
};

export default Navbar;

```

C. Homepage Component (`src/pages/Homepage.jsx`)

Fetches note items, displays card lists, handles delete queries, and shows an empty state if no notes are found.

```

import { useEffect, useState } from "react";
import { Link } from "react-router-dom";
import axios from "axios";
import toast from "react-hot-toast";
import { Edit2, Trash2, Calendar, FileText } from "lucide-react";
import Navbar from "../components/Navbar";

```

```
const Homepage = () => {
  const [notes, setNotes] = useState([]);
  const [loading, setLoading] = useState(true);

  // Fetch all notes from Backend API
  const fetchNotes = async () => {
    try {
      setLoading(true);
      const response = await axios.get("http://localhost:3000/api/notes");
      setNotes(response.data);
    } catch (error) {
      console.error("Error fetching notes:", error);
      toast.error("Failed to load notes");
    } finally {
      setLoading(false);
    }
  };

  useEffect(() => {
    fetchNotes();
  }, []);

  // Handle Delete Action
  const handleDelete = async (id) => {
    if (!window.confirm("Are you sure you want to delete this note?")) return;

    try {
      await axios.delete(`http://localhost:3000/api/notes/${id}`);
      toast.success("Note deleted successfully");
      // Remove note from local state instantly to avoid reloading page
      setNotes(notes.filter((note) => note._id !== id));
    } catch (error) {
      console.error("Error deleting note:", error);
      toast.error("Failed to delete note");
    }
  };

  return (
    <div className="min-h-screen bg-base-100 pb-12">
      <Navbar />

      <main className="max-w-6xl mx-auto px-4 py-8">
        <div className="flex items-center justify-between mb-8">
          <div>
            <h2 className="text-2xl font-bold text-base-content">My Thinkboard</h2>
            <p className="text-sm text-base-content/60">Organize your thoughts</p>
          </div>
          <div className="badge badge-primary badge-outline font-semibold">
            {notes.length} {notes.length === 1 ? "Note" : "Notes"}
          </div>
        </div>

        {/* Loading Spinner */}
        {loading ? (
          <div className="flex flex-col items-center justify-center py-20 space-y-4">
            <span className="loading loading-spinner loading-lg text-primary"></span>
            <p className="text-base-content/60 font-medium">Loading notes...</p>
          </div>
        ) : notes.length === 0 ? (
          // Empty State Design
          <div className="text-center py-20 border-2 border-dashed border-base-content/20 rounded-2xl max-w-md mx-auto px-6 bg-base-200/50">
            <div className="w-16 h-16 bg-primary/10 rounded-full flex items-center justify-center mx-auto mb-4 text-primary">
              <FileText className="size-8" />
            </div>
            <h3 className="text-lg font-bold text-base-content mb-1">No notes yet</h3>

```

```

        <p className="text-sm text-base-content/60 mb-6">Create your first note on
Thinkboard.</p>
        <Link to="/create" className="btn btn-primary px-6">Create a Note</Link>
    </div>
    ) : (
        // Note Grid Layout
        <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6">
            {notes.map((note) => (
                <div key={note._id} className="card bg-base-200 border border-base-content/10
shadow-sm hover:shadow-md hover:border-primary/30 transition-all group flex flex-col justify-
between">
                    <div className="card-body p-6">
                        <h3 className="card-title text-lg font-bold text-base-content group-hover:text-
primary transition-colors line-clamp-1">
                            {note.title}
                        </h3>
                        <p className="text-sm text-base-content/75 mt-2 line-clamp-4 whitespace-pre-
line leading-relaxed">
                            {note.content}
                        </p>
                    </div>

                    <div className="px-6 py-4 bg-base-300/40 border-t border-base-content/5 rounded-
b-2xl flex items-center justify-between mt-auto">
                        <div className="flex items-center gap-1.5 text-xs text-base-content/50 font-
medium">
                            <Calendar className="size-3.5" />
                            <span>
                                {new Date(note.createdAt).toLocaleDateString("en-US", {
                                    month: "short",
                                    day: "numeric",
                                    year: "numeric",
                                })}
                            </span>
                        </div>

                        <div className="flex items-center gap-2">
                            <Link to={`/notes/${note._id}`} className="btn btn-sm btn-ghost btn-circle
text-base-content/70 hover:text-primary hover:bg-primary/10" title="Edit Note">
                                <Edit2 className="size-4" />
                            </Link>
                            <button onClick={() => handleDelete(note._id)} className="btn btn-sm btn-
ghost btn-circle text-base-content/70 hover:text-error hover:bg-error/10" title="Delete Note">
                                <Trash2 className="size-4" />
                            </button>
                        </div>
                    </div>
                </div>
            )))
        </div>
    )}
</main>
</div>
);
};

export default Homepage;

```

D. Create Note Component (`src/pages/createpage.jsx`)

Provides the input form layout for saving new notes.

```

import { useState } from "react";
import { useNavigate, Link } from "react-router-dom";
import axios from "axios";

```

```

import toast from "react-hot-toast";
import { ArrowLeft, Save } from "lucide-react";
import Navbar from "../components/Navbar";

const CreatePage = () => {
  const [title, setTitle] = useState("");
  const [content, setContent] = useState("");
  const [isSubmitting, setIsSubmitting] = useState(false);
  const navigate = useNavigate();

  const handleSubmit = async (e) => {
    e.preventDefault();

    if (!title.trim() || !content.trim()) {
      toast.error("Please fill in all fields");
      return;
    }

    try {
      setIsSubmitting(true);
      // HTTP POST request to backend API to create a note document
      await axios.post("http://localhost:3000/api/notes", { title, content });
      toast.success("Note created successfully!");
      navigate("/"); // Redirect back to Homepage
    } catch (error) {
      console.error("Error creating note:", error);
      toast.error("Failed to create note");
    } finally {
      setIsSubmitting(false);
    }
  };

  return (
    <div className="min-h-screen bg-base-100 pb-12">
      <Navbar />
      <main className="max-w-2xl mx-auto px-4 py-8">
        <div className="mb-6">
          <Link to="/" className="inline-flex items-center gap-2 text-sm font-semibold text-base-content/60 hover:text-primary">
            <ArrowLeft className="size-4" />
            <span>Back to Thinkboard</span>
          </Link>
        </div>

        <div className="card bg-base-200 border border-base-content/10 shadow-lg">
          <div className="card-body p-6 md:p-8">
            <h2 className="card-title text-2xl font-bold mb-6">Create New Note</h2>
            <form onSubmit={handleSubmit} className="space-y-6">
              <div className="form-control w-full">
                <label className="label">
                  <span className="label-text font-semibold text-base-content/75">Title</span>
                </label>
                <input
                  type="text"
                  placeholder="Enter note title..."
                  value={title}
                  onChange={(e) => setTitle(e.target.value)}
                  className="input input-bordered w-full focus:outline-none focus:border-primary text-base-content"
                  maxLength={100}
                  required
                />
              </div>

              <div className="form-control w-full">
                <label className="label">
                  <span className="label-text font-semibold text-base-content/75">Content</span>

```

```

        </label>
        <textarea
          placeholder="Write your note contents here..."
          value={content}
          onChange={(e) => setContent(e.target.value)}
          className="textarea textarea-bordered h-64 focus:outline-none focus:border-
primary leading-relaxed text-base-content"
          required
        />
      </div>

      <div className="card-actions justify-end pt-4 gap-3">
        <Link to="/" className="btn btn-ghost">Cancel</Link>
        <button type="submit" disabled={isSubmitting} className="btn btn-primary px-6
gap-2">
          {isSubmitting ? (
            <>
              <span className="loading loading-spinner loading-sm"></span>
              <span>Creating...</span>
            </>
          ) : (
            <>
              <Save className="size-4" />
              <span>Save Note</span>
            </>
          )}
        </button>
      </div>
    </form>
  </div>
</div>
</main>
</div>
);
};

export default CreatePage;

```

E. Edit Note Component (`src/pages/NoteDetailsPage.jsx`)

Fetches the existing note data based on the route parameters (`:id`) and provides an update form.

```

import { useEffect, useState } from "react";
import { useParams, useNavigate, Link } from "react-router-dom";
import axios from "axios";
import toast from "react-hot-toast";
import { ArrowLeft, Save } from "lucide-react";
import Navbar from "../components/Navbar";

const NoteDetailsPage = () => {
  const { id } = useParams(); // Retrieves note ID from path URL
  const navigate = useNavigate();
  const [title, setTitle] = useState("");
  const [content, setContent] = useState("");
  const [loading, setLoading] = useState(true);
  const [isSubmitting, setIsSubmitting] = useState(false);

  // Fetch the note details on page mount
  useEffect(() => {
    const fetchNote = async () => {
      try {
        setLoading(true);
        const response = await axios.get(`http://localhost:3000/api/notes/${id}`);
        setTitle(response.data.title);
        setContent(response.data.content);
      }
    };
    fetchNote();
  }, [id]);

```

```

    } catch (error) {
      console.error("Error fetching note:", error);
      toast.error("Failed to load note details");
      navigate("/");
    } finally {
      setLoading(false);
    }
  };

  fetchNote();
}, [id, navigate]);

// Handle Note Update Submit
const handleSubmit = async (e) => {
  e.preventDefault();

  if (!title.trim() || !content.trim()) {
    toast.error("Please fill in all fields");
    return;
  }

  try {
    setIsSubmitting(true);
    // HTTP PUT Request to update the note details by ID
    await axios.put(`http://localhost:3000/api/notes/${id}`, { title, content });
    toast.success("Note updated successfully!");
    navigate("/"); // Redirect back to Homepage
  } catch (error) {
    console.error("Error updating note:", error);
    toast.error("Failed to update note");
  } finally {
    setIsSubmitting(false);
  }
};

return (
  <div className="min-h-screen bg-base-100 pb-12">
    <Navbar />
    <main className="max-w-2xl mx-auto px-4 py-8">
      <div className="mb-6">
        <Link to="/" className="inline-flex items-center gap-2 text-sm font-semibold text-base-content/60 hover:text-primary">
          <ArrowLeft className="size-4" />
          <span>Back to Thinkboard</span>
        </Link>
      </div>

      {loading ? (
        <div className="flex flex-col items-center justify-center py-20 space-y-4">
          <span className="loading loading-spinner loading-lg text-primary"></span>
          <p className="text-base-content/60 font-medium">Loading note details...</p>
        </div>
      ) : (
        <div className="card bg-base-200 border border-base-content/10 shadow-lg">
          <div className="card-body p-6 md:p-8">
            <h2 className="card-title text-2xl font-bold mb-6">Edit Note</h2>
            <form onSubmit={handleSubmit} className="space-y-6">
              <div className="form-control w-full">
                <label className="label">
                  <span className="label-text font-semibold text-base-content/75">Title</span>
                </label>
                <input
                  type="text"
                  placeholder="Enter note title..."
                  value={title}
                  onChange={(e) => setTitle(e.target.value)}
                  className="input input-bordered w-full focus:outline-none focus:border-

```

```

primary text-base-content"
      maxLength={100}
      required
    />
  </div>

  <div className="form-control w-full">
    <label className="label">
      <span className="label-text font-semibold text-base-content/75">Content</span>
    </label>
    <textarea
      placeholder="Write your note contents here..."
      value={content}
      onChange={(e) => setContent(e.target.value)}
      className="textarea textarea-bordered h-64 focus:outline-none focus:border-
primary leading-relaxed text-base-content"
      required
    />
  </div>

  <div className="card-actions justify-end pt-4 gap-3">
    <Link to="/" className="btn btn-ghost">Cancel</Link>
    <button type="submit" disabled={isSubmitting} className="btn btn-primary px-6
gap-2">
      {isSubmitting ? (
        <>
          <span className="loading loading-spinner loading-sm"></span>
          <span>Saving...</span>
        </>
      ) : (
        <>
          <Save className="size-4" />
          <span>Save Changes</span>
        </>
      )}
    </button>
  </div>
</form>
</div>
</div>
  </div>
  </main>
</div>
);
};

export default NoteDetailsPage;

---
```

6. API Communication & CORS

What is CORS?

CORS (Cross-Origin Resource Sharing) is a security feature enforced by web browsers.

By default, web browsers block frontend scripts running on one origin (e.g., React on `http://localhost:5173`) from reading response data from a different origin (e.g., Node server on `http://localhost:3000`).

How We Configure It

- In `backend/src/server.js`, we import the `cors` package and apply it as a global middleware:`

```
import cors from 'cors';  
app.use(cors()); // Allows all origins to access our API endpoints
```

- In the React code, we make API calls directly to the Express server address (`http://localhost:3000/api/notes`) using `axios` methods (`axios.get`, `axios.post`, etc.).

7. Running the Project Locally

To test the application locally on your computer, follow these steps:

Step 1: Backend Setup

1. Create a `.env` file in the `/backend` folder.
2. Add the database connection string and port:

```
PORT=3000  
MONGO_URI=mongodb+srv://<username>:<password>@cluster0.xxxx.mongodb.net/?  
retryWrites=true&w=majority  
NODE_ENV=development
```

3. Open a terminal, navigate to `/backend`, install dependencies and run:

```
npm install  
npm run dev
```

The terminal should output `Connected to MongoDB successfully` and `Server is running on port: 3000`.

Step 2: Frontend Setup

1. Open a second terminal window (keep the backend server running!).
2. Navigate to the `/frontend` folder and run:

```
npm install  
npm run dev
```

3. Copy the URL displayed in the terminal (usually `http://localhost:5173`) and open it in your browser.
4. Test creating a note, updating it, and deleting it.

8. Deployment to Production (Render)

Render is a unified platform to host your web services and static sites.

Option A: Hosting Fullstack Monorepo as a Single Web Service

Since the Express backend serving script is set up to host the built React static files, we can deploy the entire app as a **single Web Service**.

1. Commit your codebase to a public/private **GitHub repository**.
2. Log in to [Render.com](https://render.com/) and click **New +** -> **Web Service**.
3. Link your GitHub repository.
4. Fill in the configuration:
 - **Name**: `Thinkboard`

- **Runtime**: ``Node``
- **Root Directory**: ``backend`` (Render will search and run inside the ``/backend`` folder).
- **Build Command**: ``npm install && npm install --prefix ../frontend && npm run build --prefix ../frontend``
- **Start Command**: ``node src/server.js``

5. Click **Advanced** and add **Environment Variables**:

- ``MONGO_URI`` = ``mongodb+srv://...`` (Your MongoDB Atlas connection URL).
- ``NODE_ENV`` = ``production`` (This activates the backend code that serves the built React frontend).

6. Deploy the Web Service.

Option B: Deploying Frontend and Backend Separately

You can also host the frontend as a **Static Site** and the backend as a **Web Service**.

1. Backend Web Service:

- **Root Directory**: ``backend``
- **Build Command**: ``npm install``
- **Start Command**: ``node src/server.js``
- **Environment Variables**: ``MONGO_URI`` (your MongoDB connection)

2. Frontend Static Site:

- **Root Directory**: ``frontend``
- **Build Command**: ``npm install && npm run build``
- **Publish Directory**: ``dist`` *(Vite's build output)*
- **Note**: Remember to update the backend API URL in your React code from ``http://localhost:3000`` to the actual deployment URL of your Render Web Service!*

This concludes the comprehensive **Thinkboard** development guide. Good luck with your project demonstration!